## Daemon thread [Service Level Thread]

Daemon thread is a low- priority thread which is used to provide background maintenance.

The main purpose of of Daemon thread to provide services to the user thread.

JVM can't terminate the program till any of the non-daemon (user) thread is active, once all the user thread will be completed then JVM will automatically terminate all Daemon threads, which are running in the background to support user threads.

The example of Daemon thread is Garbage Collection thread, which is running in the background for memory management.

In order to make a thread as a Daemon thread as well as to verify whether a thread is daemon thread or not, Java software people has provided the following two methods

1) public final void setDaemon(boolean on) : If the boolean value is set to be true the thread will work as a Daemon thread.

2) public final boolean isDaemon() : Will verify whether the thread is daemon thread OR not (User thread)

```java
package com.ravi.daemon;

public class DaemonThreadDemo1
{
  public static void main(String[] args)
    {
        IO.println("Main Thread started...");

        Thread daemonThread = new Thread(()->
        {
         while(true)
         {
            IO.println("Daemon thread is running");
            try
            {
                Thread.sleep(2000);
            }
            catch(InterruptedException e)
            {
                e.printStackTrace();
            }
         }
       });

       daemonThread.setDaemon(true);
       daemonThread.start();

       Thread userThread = new Thread(()->
       {
             for(int i=1; i<=20; i++)
             {
                 IO.println(i+" by user thread");
                    try
                 {
                    Thread.sleep(1500);
                 }
                 catch(InterruptedException e)
                 {
                    e.printStackTrace();
                 }
             }
       });

      userThread.start();

       IO.println("Main Thread Ended...");
    }
 }
```

Till user thread is active, daemon thread will run in the background

---

```java
package com.ravi.daemon;

public class DaemonThread2
{
     public static void main(String[] args)
     {
         Thread t = Thread.currentThread();
         IO.println(t.isDaemon());

         t.setDaemon(true);
         System.out.println(t.isDaemon());

     }

}
```

Note : main thread is already started by JVM so, we cann't make main thread as a deamon thread otherwise
we will get an exception java.lang.IllegalThreadStateException

---

## public void interrupt() Method of Thread class :

It is a predefined non static method of Thread class. The main purpose of this method to disturb the execution of the Thread, if the thread is in waiting or sleeping state.

Whenever a thread is interupted then it throws InterruptedException so the thread (if it is in sleeping or waiting mode) will get a chance to come out from a particular logic.

Points :-

If we call interrupt method and if the thread is not in sleeping or waiting state then it will behave normally.

If we call interrupt method and if the thread is in sleeping or waiting state then we can stop the thread gracefully.

*Overall interrupt method is mainly used to interrupt the
thread safely so we can manage the resources easily.

Methods :

1) public void interrupt () :- Used to interrupt the Thread but the thread must be in sleeping or waiting mode.

2) public boolean isInterrupted() :- Used to verify whether thread is interrupted or not.

```java
package com.ravi.interrupt;

class Test implements Runnable
{
    @Override
    public void run()
    {
        boolean isInterrupted = Thread.currentThread().isInterrupted();

        IO.println("Is Thread Interrupted :"+isInterrupted);

        for(int i=1; i<=10; i++)
        {
            IO.println("i value is :"+i);
        }
    }
}

public class InterruptDemo1
{

    public static void main(String[] args)
    {
        Thread t1 = new Thread(new Test());
        t1.start();

        //main thread is interrupting the child thread
        t1.interrupt(); //isInterrupted = true;

    }

}
```

The child thread is not in a sleeping state so, even after interruption It will throw any exception

---

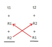
```java
package com.ravi.interrupt;

class Interrupt extends Thread
{
   @Override
   public void run()
    {
       boolean isInterrupted = Thread.currentThread().isInterrupted();
       IO.println("Is thread Interrupted :"+isInterrupted);

       for(int i=1; i<=5; i++)
         {
            System.out.println(i);

          try
           {
            Thread.sleep(5000);  //isInterrupted = false , InterruptedException

           }
           catch (InterruptedException e)
           {
               System.err.println("Thread has Interrupted");
               Thread.currentThread().interrupt(); //isInterrupted = true

           }

         }
    }
}
public class  InterruptDemo2
{
     public static void main(String[] args)
     {
        Interrupt it = new Interrupt();
         it.start();
         it.interrupt(); //main thread is interrupting the child thread

     }
}
```

---

```java
package com.ravi.interrupt;

public class InterruptDemo3
{
    public static void main(String[] args)
     {
        Thread thread = new Thread(new MyRunnable());
        thread.start();

        try
        {
         Thread.sleep(9000);  //main thread is waiting for 5 sec
        }
        catch(InterruptedException e)
        {
         e.printStackTrace();
        }

        thread.interrupt();


     }
}

class MyRunnable implements Runnable
{
     @Override
     public synchronized void run()
     {
          try
          {
               while(!Thread.currentThread().isInterrupted())
               {
                    Thread.sleep(500);
                    IO.println("Child thread is running with lock ");
               }
          }
          catch(InterruptedException e)
          {
               IO.println(e);
          }
          finally
          {
               IO.println("Resourse is released here!!!");
          }

     }


}
```

---

## Deadlock :



```
Thread t1 = new Thread();
synchronized(r1)
{
   //Blocked
   synchronized(r2)
   {
   }
}

Thread t2 = new Thread();
synchronized(r2)
{
  //Blocked
  synchronized(r1)
   {
   }
}
```

It is a situation where two or more than two threads are in blocked state forever, here threads are waiting to acquire another thread resource without releasing it's own resource.

This situation happens when multiple threads demands same resource without releasing its own attached resource so as a result we get Deadlock situation and our execution of the program will go to an infinite state as shown in the diagram.

```java
package com.ravi.interrupt;

public class DeadlockExample
{
  public static void main(String[] args)
  {
     String resource1 = new String("Ameerpet");  //(R1)
     String resource2 = new String("S R Nagar"); //(R2)

    // t1 tries to lock resource1(R1) then resource2(R2)

    Thread t1 = new Thread()
     {
      @Override
      public void run()
          {
               synchronized (resource1)
               {
                IO.println("Thread 1: locked resource 1");
                   try
                    {
                    Thread.sleep(1000);
                    }
                    catch (Exception e)
                    {}


                //t1 thread is waiting here for Lock2
               synchronized (resource2) //Nested synchronized block
                {
                System.out.println("Thread 1: locked resource 2");
                }
               }
      }
    };


    // t2 tries to lock resource2 then resource1
    Thread t2 = new Thread()
    {
      @Override
      public void run()
      {
        synchronized (resource2)
           {
          System.out.println("Thread 2: locked resource 2");
              try
             {
             Thread.sleep(1000);
             }
             catch (Exception e)
             {}

           //t2 thread will wait for L1  (Resourse1)
          synchronized (resource1) //Nested synchronized block
         {
           System.out.println("Thread 2: locked resource 1");
         }
       }
     }
    };
    t1.start();
    t2.start();
  }
}
```

---

## volatile keyword [All the threads will read the value of volatile variable directly from main memory]

While working in a multithreaded environment multiple threads can perform read and write operation with common variable/data (chances of Data inconsistency so use synchronized OR AtomicInteger) concurrently.

In order to store the value temporarly, Every thread is having local cache memory (PC Register) but if we declare a variable with volatile modifier then variable's value is not stored in a thread's local cache; it is always read from the main memory.

So the conclusion is, a volatile variable value is always read from and written directly to the main memory, which ensures that changes made by one thread are visible to all other threads immediately.

```java
package com.ravi.interrupt;

class SharedData
{
    private volatile boolean flag = false;

    public void startThread()
    {
        Thread writer = new Thread(() ->
        {
            try
            {
              IO.println("Writer thread started");
                Thread.sleep(5000);  //Writer thread will go for 5 sec waiting state
                this.flag = true;
                IO.println("Writer thread make the flag value as true");
            }
            catch (InterruptedException e)
            {
                e.printStackTrace();
            }
        });


        Thread reader = new Thread(() ->
        {
            while (!flag) //Cache memory still the value of flag is false
            {

            }
            IO.println("Reader thread got the updated value");
        });

        writer.start();
        reader.start();
    }

}

public class VolatileExample
{
    public static void main(String[] args)
    {
        new SharedData().startThread();
    }
}
```

***In btween extends Thread and implements Runnable which one is better and Why ?

Ans : Both are the ways in java.lang package to create a Thread.
In between extends Thread and implements Runnable, implements
Runnable is more better approach due to the following reasons :

a) In extends Thread we cannot extend any other class (MI is not supported by using class) but in implements Runnable we can extend only one class.

b) extends Thread provides tight coupling approach so to create multiple threads we need multiple sub class objects on the other
hand implements Runnable provides loose coupling approach so for creating multiple threads we need only sub class object.

Example :
Case 1 :

```
class Test extends Thread
{
}

Test t1 = new Test();
t1.start();
Test t2 = new Test();
t2.start();
```

Case 2:

```
class UserThread implements Runnable
{
}

UserThread ut = new UserThread();

Thread t1 = new Thread(ut);
Thread t2 = new Thread(ut);
```

3) By using extends Thread we can't use Lambda Expression but using implements we can use Lambda Expression

4)If a class extends from Thread class then all the Thread class methdos are available in the sub class but if we use implements  Runnable then Thread class methos are not available.